

## Unterschied Referenz und Objekt:

Wir definieren in

```
private final Collection<FileSystemElement> elements = new LinkedList<>();
```

eine Liste als final. Was bedeutet das?

Wenn du eine Variable als `final` deklarierst, bedeutet das lediglich, dass **die Variable selbst** (der Zeiger) nicht neu zugewiesen werden kann. Es bedeutet **nicht**, dass das Objekt, auf das sie zeigt, unveränderlich ist.

Man kann sich das wie folgt vorstellen:

- **Die Variable (`elements`):** Das ist wie ein Wegweiser, der im Boden einbetoniert ist (`final`). Du kannst den Wegweiser nicht ausreißen und woanders hinstellen (auf eine andere Liste zeigen lassen).
- **Das Objekt (`LinkedList`):** Das ist das Haus, auf das der Wegweiser zeigt. Du kannst das Haus zwar nicht austauschen, aber du kannst problemlos **Dinge hineinstellen oder herausnehmen** (Elemente hinzufügen oder entfernen).

### Erlaubt: Den Zustand des Objekts ändern

Das Objekt hinter `elements` ist eine `LinkedList`. Diese ist "mutable" (veränderbar). Methoden wie `.add()`, `.remove()` oder `.clear()` ändern den **Inhalt** der Liste, nicht die Referenzvariable selbst.

```
dir1 = new Directory("dir1");
file1 = new File("file1", 5);
dir1.add(file1);
dir2 = new Directory("dir2");
dir1.add(dir2);
dir3 = new Directory("dir3");
dir2.add(dir3);
dir4 = new Directory("dir4");
dir2.add(dir4);
file2 = new File("file2", 10);
dir3.add(file2);
file3 = new File("file3", 20);
dir3.add(file3);
```

### Verboten: Die Referenz ändern

Da `elements` als `final` markiert ist, darfst du der Variable kein *neues* Objekt zuweisen, nachdem sie einmal initialisiert wurde.

```
dir1.elements = new ArrayList<>(); // FEHLER! Variable ist final.
dir1.elements = null;              // FEHLER! Variable ist final.
```

## Wann wäre das Hinzufügen verboten?

Das Hinzufügen wäre nur verboten, wenn die **Liste selbst** unveränderlich wäre (Immutable), nicht wegen des `final` Keywords der Variable.

```
private final Collection<String> elements =  
Collections.unmodifiableList(new LinkedList<>());
```

## Warum ist das hier überhaupt `final`?

Da die Variable `final` ist, muss sie zwingend initialisiert werden (entweder direkt bei der Deklaration oder im Konstruktor).

Das bedeutet, dass du im Rest der Klasse niemals prüfen musst: `if (elements != null)`. Du kannst dich zu 100% darauf verlassen, dass die Liste existiert. Das macht den Code viel sauberer und robuster.